

TECHNICAL PROJECT REPORT

CIA-3 Component 2: Micro Project

CSEAM731-REINFORCEMENT LEARNING

Dynamic Pricing on an E-commerce Platform with Deep Reinforcement Learning: A Measured Comparison of DQN, DDPG, PPO and SAC on a Shared Pricing MDP

Submitted by

[Member 1 – Register Number]

[Member 2 – Register Number]

[Member 3 – Register Number]

[Member 4 – Register Number]

Class: 7BTCS AIML – [A/B/C]

Course: Reinforcement Learning

Institution: CHRIST (Deemed to be University), Bengaluru

Academic Year: 2026–2027

Abstract

Dynamic pricing asks a seller to re-set a product's price every period as demand, traffic and competitors change. This report reproduces, in a controlled and measurable form, the comparison behind Liu et al.'s field-tested Tmall pricing system [1]: four reinforcement-learning algorithms — DQN, DDPG, PPO and SAC — are trained on one identical Markov decision process (a 35-feature product-month state, a price move within $\pm 25\%$, and the Difference of Revenue Conversion Rate as reward) and evaluated on 20 held-out products they never saw. Because logged sales data only reveal the outcome at the one price actually charged, a counterfactual demand-response model (a within-product log-log elasticity baseline plus a capped gradient-boosted correction, holdout $R^2 = 0.94$) serves as the environment's transition and reward function, with guard rails against exploitation. All four learners were validated on standard Gymnasium tasks before use and trained under an identical budget of 15,000 environment steps for each of 3 seeds. SAC achieved the best held-out DRCR (0.0822 ± 0.0009) and a +3.9 % simulated revenue gain over the seller's own prices, followed by PPO (0.0778) and DDPG (0.0748); DQN (0.0313) fell below the no-change baseline, a result attributable to both its discretised action space and an exploration schedule longer than the training budget. Continuous-action, entropy-regularised learning is the most effective and stable choice for this problem.

Keywords: dynamic pricing, reinforcement learning, Deep Q-Network, DDPG, PPO, Soft Actor-Critic, demand modelling, e-commerce

1. Introduction

1.1 Background

Online marketplaces list millions of products whose demand shifts daily with seasonality, promotions, customer traffic and competitors' prices. A price set once and left alone is wrong most of the time: too high and units are lost, too low and margin is given away. Large platforms therefore price dynamically, re-setting each product's price at regular intervals. Alibaba's supply-chain team built such a system with deep reinforcement learning and deployed it on Tmall.com, China's largest B2C marketplace, from July 2018; in a 30-day field experiment the RL policies out-earned manual pricing by a wide margin, with the continuous-action agent (DDPG) scoring a Difference of Revenue Conversion Rate of 6.07 against a manual baseline rescaled to 1.00, and the discrete agent (DQN) 5.03 [1].

1.2 Motivation

Each month's price does not only earn that month's revenue: it becomes the reference the customer and the competitor react to next month, it changes the product's rolling sales record, and it moves the product's position in the category's price ranking. The decision is therefore sequential — today's choice changes tomorrow's state — and the objective is cumulative. Rules of thumb (“undercut the cheapest competitor by 2 %”) cannot see this; they optimise one period at a time and cannot adapt when the relationship between price and demand changes. The team's Component-1 report studied four RL algorithms on this problem conceptually; this Component-2 project implements all four, measures them under a fair protocol, and reports what actually happened.

1.3 Why Reinforcement Learning?

The problem maps cleanly onto the RL formalism of Sutton and Barto [8]. The agent is the pricing policy for one product. The environment is the market: customer traffic, competitor prices and seasonality that the agent observes but does not control, plus the demand response to the agent's own price. The state is what is known about the product before this month's price is set. The action is the new price. The reward is the change in traffic-normalised conversion (DRCR), the same signal used in [1]. The long-term objective is the discounted

sum of these improvements, so the agent is rewarded for prices that keep paying off, not for one-off spikes. Trial-and-error learning from outcomes is exactly what is needed when the demand function is unknown and non-stationary.

1.4 Selected RL Approaches

- **DQN** (Deep Q-Network [4]): value-based, off-policy; chooses among $K = 11$ discrete price buckets. The paper's own baseline.
- **DDPG** (Deep Deterministic Policy Gradient [5]): actor-critic, off-policy; outputs an exact continuous price. The paper's field winner.
- **PPO** (Proximal Policy Optimization [6]): stochastic actor-critic, on-policy; clipped updates chosen for robustness to a noisy reward.
- **SAC** (Soft Actor-Critic [7]): maximum-entropy actor-critic, off-policy; twin critics and a self-tuned exploration temperature.

1.5 Objectives

- Build a reproducible pricing environment from logged product-month data, including a validated counterfactual demand model with safeguards against reward hacking.
- Implement DQN, DDPG, PPO and SAC from first principles in PyTorch behind one common agent interface, and verify each on a standard benchmark before use.
- Train all four under an identical budget (15,000 environment steps $\times$ 3 seeds) and evaluate them deterministically on 20 held-out products.
- Compare them on at least four metrics — held-out DRCR, cross-seed stability, simulated revenue uplift over the seller, convergence speed, guard-rail behaviour and computational cost — and identify the best approach with evidence.
- Deliver an interactive dashboard and a live application so the results can be inspected product by product.

1.6 Report Organization

Section 2 reviews related work. Section 3 defines the case study as a sequential decision problem. Section 4 specifies the state, action, reward, episode and environment characteristics. Section 5 presents each algorithm and its implementation. Section 6 documents the experimental setup and hyperparameters. Section 7 reports results with figures and tables; Section 8 compares the approaches on six metrics; Section 9 interprets them and states limitations. Section 10 proposes an extension, Section 11 concludes, and the appendices give project resources and additional results.

2. Related Work

Reinforcement learning has been applied to pricing for over a decade, moving from tabular methods on stylised inventory problems to deep methods on live marketplaces. Table 1 summarises the studies that shaped this project.

Table 1. Related studies on reinforcement learning for pricing and the algorithms compared in this project.

Study	Problem / Domain	RL Approach	Key Finding	Limitation
Liu et al., 2019 [1]	Dynamic pricing of thousands of SKUs on Tmall.com (field experiment)	DQN (discrete buckets) and DDPG (continuous price); DRRCR reward; pre-training on human prices	Both RL policies beat manual pricing in 30-day live tests; DDPG (6.07) > DQN (5.03) vs. manual = 1.00	Data and simulator are proprietary; only two algorithms compared; low-traffic SKUs need pre-training
Kastius & Schlosser, 2022 [2]	Pricing under duopoly / oligopoly competition (simulation)	DQN and SAC	Both learn reasonable policies; SAC outperforms DQN; RL sellers can be pushed into tacit	Synthetic market models; no field validation

			collusion	
Rana & Oliveira, 2014 [3]	Selling fixed inventory by a deadline (airlines, hotels, fashion)	Q-learning and $Q(\lambda)$ with eligibility traces	Model-free RL learns near-optimal pricing without a known demand function and tracks non-stationarity	Tabular; small state spaces; single seller
Mnih et al., 2015 [4]	General control from high-dimensional input (Atari)	DQN: replay buffer + target network	Stable deep value learning over discrete actions	Discrete actions only; price precision limited by bucket count
Lillicrap et al., 2015 [5]	Continuous control benchmarks	DDPG: deterministic actor + critic with target networks	Extends DQN's stabilisers to continuous actions	Sensitive to hyperparameters and to the external exploration noise schedule

2.1 Project Gap

Liu et al. compare only DQN and DDPG, on data nobody else can access. Kastius and Schlosser compare DQN and SAC, but in a synthetic competition model. No study places all four algorithm families — discrete value-based, deterministic continuous, on-policy clipped, and entropy-regularised — on one identical, data-driven pricing MDP with held-out evaluation and multiple seeds. This project fills that gap and, additionally, makes the counterfactual demand model that any such comparison depends on explicit, validated and guarded.

3. Problem Definition and Case Study

3.1 Real-Time Problem

A marketplace seller lists a product in a category alongside three comparable competitors. Every month the seller must set the product's price for the coming month knowing its own attributes, its recent sales and traffic, its price history and how it compares with the competitors. The goal is to maximise revenue conversion over time, not to win any single month. This is the setting of [1]; the data used here follow the Kaggle Retail Price Optimization schema [14] (one row per product-month with price, units, freight, score and three competitor prices).

3.2 Sequential Decision-Making

The price chosen in month t becomes the entering price in month $t+1$, so the allowed move ($\pm 25\%$) is anchored to the previous decision. Units sold in month t enter the rolling three-month sales feature that the agent sees in months $t+1$ to $t+3$. The reward itself is a difference between consecutive months' conversion rates, so a spike bought by a deep cut is immediately paid back the next month. A myopic policy that maximises this month's revenue therefore differs from the policy that maximises the discounted sum — which is precisely the case for RL.

3.3 Agent and Environment

Agent: one pricing policy per algorithm, applied to every product (the state carries the product's identity through its features and category, so one network serves all products).

Environment: PricingEnv, a Gymnasium [11] environment that replays a product's real month-by-month context (competitor prices, category traffic, seasonality) and uses a fitted demand model to compute how many units sell at whatever price the agent chooses. 60 products are used for training and 20 are held out for evaluation.

3.4 Decisions and Long-Term Objective

Each decision is a new price within $\pm 25\%$ of the entering price. The immediate reward is $100 \times \text{DRCR}$, where $\text{DRCR} = \text{RCR}(t) - \text{RCR}(t-1)$ and $\text{RCR} = \text{revenue} / \text{category traffic}$. The long-term objective is the expected

discounted return with $\gamma = 0.99$ over the product's horizon, which rewards sustained improvement in conversion rather than a single good month.

3.5 Limitations of Rule-Based Systems

Fixed rules (cost-plus, match-the-median, undercut-by-x %) share three weaknesses. They ignore the product's own history, so they cannot tell a product whose demand is falling from one whose traffic is falling. They are one-step: they do not account for how this month's price constrains next month's move or changes the competitor's response. And they must be re-tuned by hand whenever elasticity, seasonality or the competitor set changes. An RL policy learns the mapping from the full state to price from outcomes and can be retrained as the market drifts.

4. RL Environment Design

4.1 State Representation

The state is a 35-dimensional vector: 25 continuous features standardised with statistics from the training products only, plus a 10-way one-hot category indicator. Every feature is computable before the month's price is set; rolling performance features are shifted by one month so that this month's outcome never leaks into the state (Table 2).

Table 2. State variables of the pricing MDP.

State Variable	Description	Type / Range	Justification
lag_price, lag_price_2	price entering this month and the month before	continuous, \$	anchor for the ± 25 % move; captures own pricing history
price_change_pct	$(\text{lag_price} - \text{lag_price_2}) / \text{lag_price_2}$	continuous, ≈ -0.3 to 0.3	direction and size of the last move
product_score, freight_price	customer rating; shipping cost	1–5; \$	own attributes that shift demand independent of price
qty_roll3, revenue_roll3, traffic_roll3	3-month rolling means of units, revenue and category traffic (previous months)	continuous	recent sales/traffic performance, as in [1]
prev_drcr	last month's DRCR	continuous	recent trend of the reward signal
price_gap_1..3, price_gap_mean	entering price minus each competitor's price, and their mean	continuous, \$	competitiveness; recomputed each month
score_gap_1..3, freight_gap_1..3	own score / freight minus competitors'	continuous	non-price competitiveness
price_rank	rank of entering price among the four prices	0 (cheapest) – 1 (dearest)	relative position in the category
month_sin, month_cos, weekday, weekend, holiday	seasonality and calendar composition	cyclic / counts / 0-1	demand seasonality
category (one-hot)	product category	10 binary	lets one network price all categories

4.2 Action Space

The action is the next month's price expressed as a move from the entering price, bounded to ± 25 %. For DQN the action space is Discrete(11): bucket k maps to a multiplier $1 + (-25 \% + 5 \% \cdot k)$, so the middle bucket is “no change” and the finest move is 5 %. For DDPG, PPO and SAC the action space is Box(−1, 1): a tanh-bounded scalar a maps to multiplier $1 + 0.25 \cdot a$, an exact price. Two constraints apply to every agent. First, the price is clipped to $[0.5 \times, 1.5 \times]$ of the product's historically observed price range (the “price guard”), and the clip is logged. Second, competitor gaps are recomputed at the chosen price, so a counterfactual price correctly changes the product's competitive position. PPO is run in continuous mode so that the three actor-critic methods are compared on equal price precision.

4.3 Reward Design

Raw revenue is a poor reward because category traffic fluctuates for reasons unrelated to price [1]. Following the paper, revenue is normalised by traffic and the reward is the month-on-month change:

$$\begin{aligned} \text{RCR}(t) &= \text{revenue}(t) / \text{traffic}(t), & \text{DRCR}(t) &= \text{RCR}(t) - \text{RCR}(t - 1) \\ r(t) &= 100 \times \text{DRCR}(t) \end{aligned}$$

Traffic is proxied by the total units sold across all products in the category that month (the dataset has no visit counts). The factor 100 keeps rewards in a range neural networks train on comfortably. Because the reward is a difference, even an excellent policy's reward averages near zero once conversion has been raised; the level it settles at and how tightly it holds there are what distinguish policies. Table 3 lists the outcomes the reward rewards and punishes.

Table 3. Reward design: situations, outcomes and the resulting reward.

Situation	Outcome	Reward	Reason
Price raised, demand nearly inelastic	revenue and RCR rise	positive (typically +5 to +30)	improvement in conversion is the objective
Price raised too far	units collapse, RCR falls	negative	over-pricing is penalised the same month
Price cut when demand is inelastic	units rise slightly, revenue falls	negative	discounting that does not pay is discouraged
Price held after a good month	RCR unchanged	≈ 0	no improvement, no penalty: stability is neutral
Deep cut for a one-off spike	RCR jumps, then falls back	positive then negative	the difference form pays the spike back next month
Price outside the guard	clipped to the guard, outcome computed there	as for the clipped price; logged	prevents exploiting the demand model off-distribution

The demand model that produces $\text{revenue}(t)$ at any candidate price is the core of the environment. It is a two-layer model fitted on the full panel: (i) a fixed-effects log-log elasticity baseline, $\log(1 + \text{qty}) = \alpha_i + \beta \cdot \log(\text{price}) + \text{controls}$, fitted after demeaning every regressor within product so that β is identified from each product's own price moves (a pooled fit gave a positive, confounded coefficient); and (ii) a gradient-boosted residual (depth 3, ≤ 250 trees, early-stopped) capturing competitor and seasonal interactions, whose output is capped at $\pm 3\sigma$ of its in-sample spread so it can bend but never flip the demand curve. On a time-based holdout (each product's last two months) it reaches $R^2 = 0.939$ and $\text{MAE} = 0.119$ in log-units; the fitted elasticity is $\beta = -0.89$. Figure 1 shows its response curves for one held-out product together with the prices the trained agents chose.

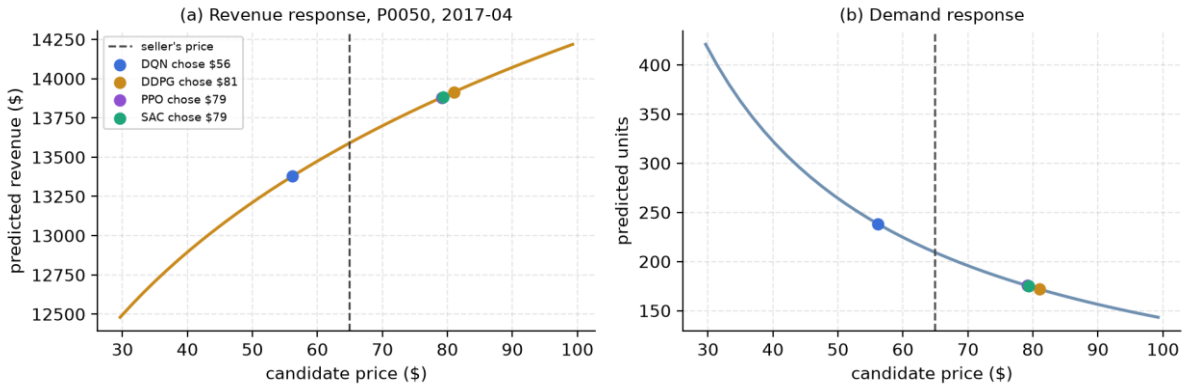


Figure 1. Demand-model response for held-out product P0050 in its first decision month: (a) predicted revenue and (b) predicted units against candidate price, with the seller's actual price (dashed) and each trained agent's chosen price (dots).

4.4 Episode Definition

An episode is one product priced month by month. Each product has 15 months of data; the first three are warm-up (they seed the rolling features and the RCR history) and the agent prices months 4–15, twelve decisions. In training, an episode draws a random training product and runs for a horizon of 20 steps; once the real months are exhausted the exogenous context (competitor prices, traffic) is extended by bootstrapping small perturbations ($\pm 3\%$ and $\pm 5\%$) of the last real row while the agent's own history keeps rolling forward — a sample-efficiency

device with only twelve real decisions per product. In evaluation, episodes run each held-out product once, over its real months only, with exploration switched off; no reported number uses bootstrapped months.

4.5 Environment Characteristics

- **Episodic / Continuing:** episodic — one product's horizon; training episodes are 20 steps, evaluation episodes 12.
- **Static / Dynamic:** dynamic — competitor prices, traffic and seasonality change every step, and the agent's own price alters the next state.
- **Deterministic / Stochastic:** the demand model is deterministic given the state and price; stochasticity enters through product sampling, the exogenous market series, and (in training) the bootstrap noise and exploration.
- **Fully / Partially Observable:** partially observable — the agent sees a 35-feature summary, not the demand function, the competitors' intentions or future traffic.
- **State-space type:** continuous, 35-dimensional (25 standardised real features + 10-way one-hot).
- **Action-space type:** discrete (11 buckets) for DQN; continuous Box(-1, 1) for DDPG, PPO and SAC; both bounded to $\pm 25\%$ of the entering price and guarded.
- **Online / Offline / Simulation-based:** simulation-based learning in a data-driven simulator: the market context is replayed from logged data and the demand response is a model fitted on that data. Learning is online with respect to the simulator.

4.6 RL Workflow

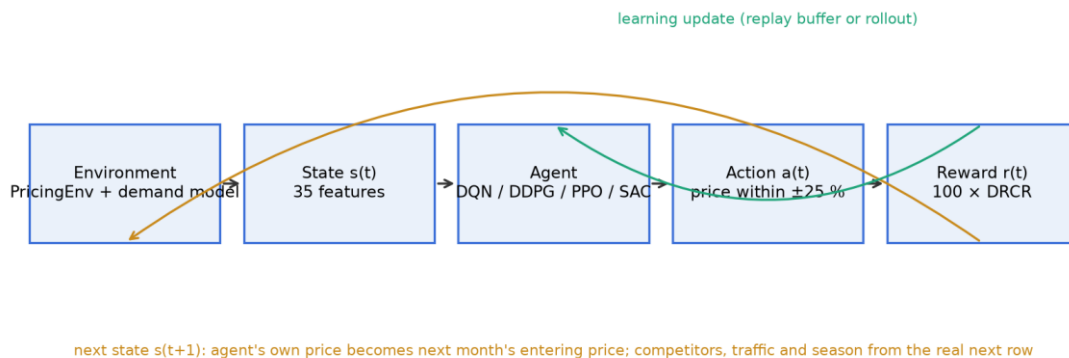


Figure 2. RL workflow for the pricing case study: the agent observes the product-month state, sets a price, the demand model returns units and revenue, the DRCR reward is computed, and the agent's own price rolls into the next state.

5. RL Approaches and Implementation

All four agents implement one interface (select_action, observe, update, save/load) so the training and evaluation loops are identical code. Every network is a two-hidden-layer MLP of 128 ReLU units trained with Adam; $\gamma = 0.99$ throughout. The pseudocode below uses s , a , r , s' for a transition and θ^- for a target network.

5.1 Deep Q-Network (DQN)

Algorithm Overview

DQN [4] approximates the action-value function $Q(s, a | \theta)$ with a neural network over a finite action set and trains it by temporal-difference learning towards the Bellman target:

$$y = r + \gamma \cdot \max_{a'} Q(s', a' | \theta^-), \quad L(\theta) = \text{Huber}(Q(s, a | \theta) - y)$$

Two mechanisms make this stable: an experience replay buffer sampled at random to decorrelate updates, and a periodically refreshed target network θ^- so the target does not move on every gradient step.

Suitability to the Case Study

Pricing as a choice among K discount tiers is a natural Q-learning problem and is the formulation Liu et al. deployed first [1]. DQN is the simplest and most stable of the four and serves as the value-based baseline; its structural cost is the bucket grid, whose resolution caps price precision.

Implementation Details

Representation: Q-network $35 \rightarrow 128 \rightarrow 128 \rightarrow 11$. Training: after 500 warm-up steps, one gradient step per environment step on a batch of 64 from a 50,000-transition buffer; Huber loss; hard target update every 100 updates. Exploration: ϵ -greedy with ϵ decayed linearly from 1.0 to 0.05 over 25,000 steps; $\epsilon = 0$ in evaluation. Update mechanism: single-step TD with the max over the target network.

Workflow / Pseudocode

1. initialise $Q(\cdot|\theta)$, copy to θ^- , empty buffer D
2. for each step: with prob. ϵ pick a random bucket, else $a = \text{argmax } Q(s, \cdot|\theta)$; step env; store $(s, a, r, s', \text{done})$ in D
3. if $|D| \geq \text{warm-up}$: sample batch; $y = r + \gamma(1-\text{done}) \max Q(s', \cdot|\theta^-)$; minimise $\text{Huber}(Q(s,a|\theta) - y)$
4. every 100 updates: $\theta^- \leftarrow \theta$; decay ϵ

Key Hyperparameters

Table 4. DQN hyperparameters.

Parameter	Value	Justification
K (price buckets)	11 over $\pm 25\%$	5 % resolution; middle bucket = no change
Learning rate	1e-3	standard for DQN on small MLPs
Replay buffer / batch	50,000 / 64	decorrelated updates; buffer covers the whole run
Target update	every 100 updates (hard)	stable bootstrapped target
ϵ schedule	1.0 $\rightarrow$ 0.05 over 25,000 steps	long exploration for a noisy reward (see §9.5: exceeds the 15k budget used)
γ	0.99	long-horizon credit for sustained conversion gains

5.2 Deep Deterministic Policy Gradient (DDPG)

Algorithm Overview

DDPG [5] is an off-policy actor-critic for continuous actions. A deterministic actor $\mu(s|\theta^u)$ outputs the price; a critic $Q(s, a|\theta^v)$ scores it. Both have slowly updated target copies.

$$\text{critic: } y = r + \gamma Q'(s', \mu'(s')), \quad L = \text{MSE}(Q(s, a) - y) \quad \text{actor: } \nabla J \approx E[\nabla_a Q(s, a) |_{a=\mu(s)} \cdot \nabla_{\theta} \mu(s)]$$

$$\text{targets: } \theta' \leftarrow \tau \theta + (1 - \tau) \theta' \quad \text{after every update}$$

Suitability to the Case Study

Price is continuous; DDPG removes the bucket trade-off by outputting an exact price, and it is the algorithm that won Liu et al.'s field test [1]. Its known weaknesses — sensitivity to hyperparameters and to the hand-scheduled exploration noise — are exactly what SAC is included to test against.

Implementation Details

Representation: actor $35 \rightarrow 128 \rightarrow 128 \rightarrow 1$ with tanh output; critic $(35+1) \rightarrow 128 \rightarrow 128 \rightarrow 1$. Training: after 500 warm-up steps, one critic and one actor update per environment step on batches of 64 from a 50,000 buffer; Polyak averaging with $\tau = 0.005$ after every update. Exploration: Ornstein–Uhlenbeck noise ($\theta = 0.15$) added to the actor output, scale annealed from 0.20 to 0.05 over 25,000 steps and reset at episode ends; no noise in evaluation.

Workflow / Pseudocode

5. initialise actor μ , critic Q and target copies μ' , Q' ; empty buffer D ; OU noise state $n = 0$
6. for each step: $a = \text{clip}(\mu(s) + n)$, $n \leftarrow \text{OU}(n)$; step env; store transition
7. sample batch; $y = r + \gamma(1-\text{done}) Q'(s', \mu'(s'))$; minimise $\text{MSE}(Q(s,a) - y)$
8. maximise $Q(s, \mu(s))$ w.r.t. actor; soft-update μ' , Q'

Key Hyperparameters

Table 5. DDPG hyperparameters.

Parameter	Value	Justification
Actor / critic learning rate	$1e-4 / 1e-3$	critic learns faster so the actor follows a settled value estimate
Soft-update τ	0.005	slow-moving targets for stability
Replay buffer / batch	50,000 / 64	as DQN, for a like-for-like off-policy setup
OU noise θ , σ	0.15; $0.20 \rightarrow 0.05$ over 25,000 steps	temporally correlated exploration of nearby prices, annealed
γ	0.99	shared across all agents

5.3 Proximal Policy Optimization (PPO)

Algorithm Overview

PPO [6] is an on-policy actor-critic that learns a stochastic policy $\pi\theta(a|s)$ and improves it by a bounded amount per batch through a clipped surrogate objective, with advantages from Generalised Advantage Estimation [10]:

$$r_t(\theta) = \pi\theta(a_t|s_t) / \pi\theta_{old}(a_t|s_t), \quad L^{CLIP} = E[\min(r_t \hat{A}_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) \hat{A}_t)]$$
$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \quad \hat{A}_t = \sum_k (\gamma\lambda)^k \delta_{t+k}$$

Suitability to the Case Study

DRCR is noisy by construction: traffic and competitor shocks move it month to month regardless of the price. PPO's clip bounds how far one unusually good or bad batch can move the policy, which the team's Component-1 analysis identified as the desirable property. The known cost is sample efficiency: PPO cannot reuse old data.

Implementation Details

Representation: Gaussian policy with a state-dependent mean ($35 \rightarrow 128 \rightarrow 128 \rightarrow 1$) and a single learned log-standard-deviation; separate value network $V(s)$. Training: collect 1,024 on-policy steps, compute GAE with $\lambda = 0.95$ (bootstrapping the last state's value if the rollout ends mid-episode), normalise advantages, then 10 epochs of minibatch (64) updates and discard the rollout. Actor and critic use separate Adam optimisers and separate gradient-norm clipping (0.5): a single combined clip lets the much larger value-loss gradients crush the policy gradient, a bug found during the smoke tests. Exploration: sampling from the policy plus an entropy bonus of 0.01; the mean action is used in evaluation. Actions are sampled unclipped and clipped to $[-1, 1]$ at the environment boundary, as in reference implementations.

Workflow / Pseudocode

9. initialise policy $\pi\theta$ (mean net + log σ), value net $V\phi$
10. repeat: run $\pi\theta$ for 1,024 steps, storing s , a , log $\pi(a|s)$, r , $V(s)$, done

11. compute $\hat{A}$ by $GAE(\gamma, \lambda)$ and returns $R = \hat{A} + V$; normalise $\hat{A}$
12. for 10 epochs over minibatches: update θ on $-L^{CLIP} - 0.01 \cdot \text{entropy}$; update ϕ on $MSE(V(s) - R)$; clip each gradient norm to 0.5
13. discard the rollout

Key Hyperparameters

Table 6. PPO hyperparameters.

Parameter	Value	Justification
Clip ϵ	0.2	standard trust-region width
Rollout / epochs / minibatch	1,024 / 10 / 64	several product episodes per update; enough passes to use each rollout
GAE λ	0.95	bias-variance balance for advantages
Learning rate	3e-4	standard for PPO
Entropy / value coefficients	0.01 / 0.5	keep exploration alive; weight the critic loss
Gradient-norm clip	0.5 (actor and critic separately)	stability; separate clipping preserves the policy gradient

5.4 Soft Actor-Critic (SAC)

Algorithm Overview

SAC [7] maximises expected return plus policy entropy, $J(\pi) = E[\sum_t r(s_t, a_t) + \alpha H(\pi(\cdot|s_t))]$, with a stochastic tanh-squashed Gaussian policy, twin critics whose minimum is used to counter overestimation, and a temperature α learned to hold the policy's entropy at a target:

$$\begin{aligned} \text{critics: } y &= r + \gamma [\min(Q_1', Q_2')(s', a') - \alpha \log \pi(a'|s')], \quad a' \sim \pi(\cdot|s') \\ \text{actor: } L &= E[\alpha \log \pi(a|s) - \min(Q_1, Q_2)(s, a)], \quad a \text{ reparameterised}; \quad \alpha: L = - E[\\ &\quad \log \alpha (\log \pi(a|s) + H_{\text{target}})] \end{aligned}$$

Suitability to the Case Study

Liu et al. flag low-traffic products as a cold-start problem solved by pre-training on human prices [1]. SAC's entropy term keeps it exploring nearby prices for as long as it is uncertain, without a hand-scheduled noise process, which is attractive precisely for data-poor products. It also tests whether DDPG's exact-price advantage can be kept while improving stability; Kastius and Schlosser report SAC beating DQN in competitive pricing [2].

Implementation Details

Representation: policy $35 \rightarrow 128 \rightarrow 128 \rightarrow (\text{mean}, \log \sigma)$, action = $\tanh(u)$ with the log-probability corrected by $-\Sigma \log(1 - \tanh^2 u + 10^{-6})$; two independent critics $(35+1) \rightarrow 128 \rightarrow 128 \rightarrow 1$ with Polyak targets. Training: after 500 warm-up steps, one update per environment step on batches of 64 from a 50,000 buffer, in the order critics $\rightarrow$ actor $\rightarrow$ temperature $\rightarrow$ target critics; α is detached in the critic and actor losses so only its own loss moves it. Target entropy is $-(\text{action dimension}) = -1$. Exploration: intrinsic — actions are sampled during training; $\tanh(\text{mean})$ is used in evaluation.

Workflow / Pseudocode

14. initialise policy π , critics Q_1, Q_2 and targets Q_1', Q_2' , $\log \alpha = 0$, buffer D
15. for each step: $a \sim \pi(\cdot|s)$ (reparameterised, tanh-squashed); step env; store transition
16. sample batch; $a' \sim \pi(\cdot|s')$; $y = r + \gamma(1-\text{done})[\min(Q_1', Q_2')(s', a') - \alpha \log \pi(a'|s')]$; minimise MSE for both critics
17. sample $\tilde{a} \sim \pi(\cdot|s)$; minimise $\alpha \log \pi(\tilde{a}|s) - \min(Q_1, Q_2)(s, \tilde{a})$ w.r.t. the policy
18. minimise $-\log \alpha (\log \pi(\tilde{a}|s) + H_{\text{target}})$ w.r.t. $\log \alpha$; soft-update Q_1', Q_2'

Key Hyperparameters

Table 7. SAC hyperparameters.

Parameter	Value	Justification
Learning rates (actor, critics, α)	3e-4 each	standard SAC setting
Soft-update τ	0.005	as DDPG
Replay buffer / batch	50,000 / 64	as DQN and DDPG
Target entropy	-1 (= -action dim)	standard heuristic [7]; α adapts automatically
Initial α	1.0 (log $\alpha = 0$)	learned from the first update onwards
γ	0.99	shared

6. Experimental Setup

6.1 Hardware

All experiments ran on a laptop: Intel Core i7-8665U (4 cores / 8 threads, 1.90 GHz base), 16 GB RAM, no discrete GPU (integrated Intel UHD 620; PyTorch CPU build, single-threaded), Windows 11 Pro. A single environment step costs 3.1 ms, dominated by the demand model.

6.2 Software

Python 3.11.9 in Visual Studio Code; PyTorch 2.14 (CPU) for all networks; Gymnasium 1.3 for the environment interface; a React 18 + Recharts 2 dashboard built with Vite 5, and a Streamlit application for live inspection.

6.3 Libraries

numpy 1.26, pandas 2.2, scikit-learn 1.9 (ridge regression and HistGradientBoostingRegressor for the demand model), torch 2.14, gymnasium 1.3, matplotlib 3.11, pyyaml, joblib, pytest 8.3, streamlit; python-pptx and python-docx for the deliverables.

6.4 Environment Configuration

Panel of 80 products in 10 categories $\times$ 15 months (1,200 rows) following the Kaggle Retail Price Optimization schema [14]; generated synthetically with an explicit demand law (category elasticities 1.0–2.5, seasonality, 3 %/month price random walk) because the sandbox had no internet access, and swappable for the real CSV without code changes. State: 35 continuous features; actions and reward as in Section 4. Product split: 60 training / 20 held-out, stratified by category. Terminal condition: 20 steps in training (with bootstrap extension after month 15), exhaustion of the 12 real decision months in evaluation. Demand model fitted on all 80 products with a time-based holdout; feature scaler fitted on training products only.

6.5 Training Configuration

Each (algorithm, seed) run: 15,000 environment steps = 750 training episodes of 20 steps; seeds 0, 1, 2 for every algorithm and for the Static and Random baselines (18 runs in total). Off-policy agents learn from step 500 with one gradient update per step; PPO updates after every 1,024-step rollout (14 updates in the budget). Evaluation: the saved checkpoint prices each of the 20 held-out products deterministically over its 12 real months; metrics are averaged over products and reported as mean $\pm$ standard deviation across seeds.

6.6 Hyperparameters

Table 8. Training hyperparameters common table. Network 2×128 ReLU, Adam, batch 64, buffer 50,000 (off-policy) for all.

Approach	Learning Rate	Gamma	Exploration	Episodes (steps)
DQN	1e-3	0.99	ϵ -greedy 1.0 $\rightarrow$ 0.05 over 25k steps	750 (15,000)

DDPG	1e-4 actor / 1e-3 critic	0.99	OU noise σ 0.20 $\rightarrow$ 0.05 over 25k steps	750 (15,000)
PPO	3e-4	0.99	stochastic policy + entropy 0.01; clip 0.2	750 (15,000)
SAC	3e-4 (actor, critics, α)	0.99	entropy-regularised, target $H = -1$, α learned	750 (15,000)

7. Results

All numbers below are measured: 3 seeds $\times$ 15,000 steps per algorithm, evaluated on the 20 held-out products; source files are results/comparison_summary.csv and the per-run logs.

7.1 Learning Curves

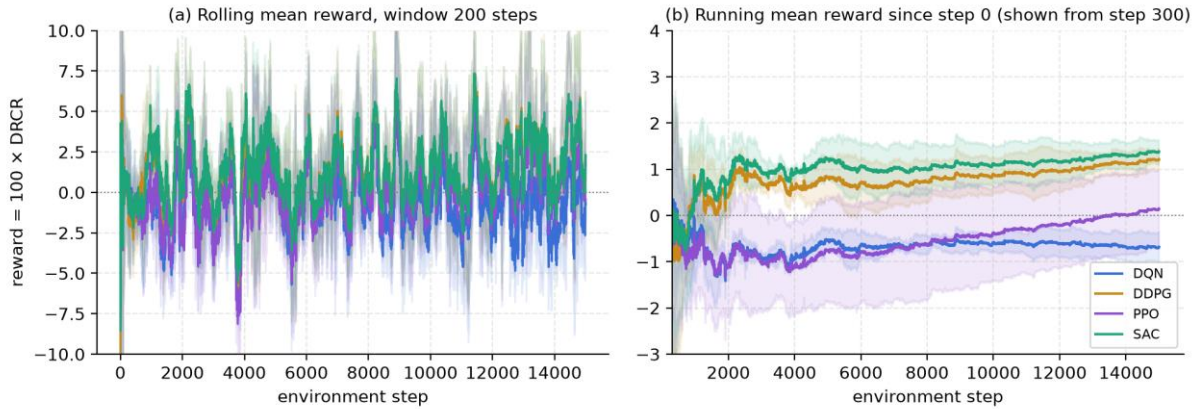


Figure 3. Learning curves, mean $\pm$ std across seeds: (a) rolling mean reward over 200 steps; (b) running mean reward since the start of training.

Panel (a) is dominated by product-level shocks: each training episode is a different product with its own traffic and competitor history, so the short-term reward is noisy for every algorithm. Panel (b) averages the noise out. SAC and DDPG rise above zero within the first 2,000 steps and settle at the top (final running means 1.39 and 1.22); PPO starts lowest, climbs steadily and is still improving at 15,000 steps (0.15), the signature of an on-policy learner that discards its data after each update; DQN stays below zero throughout (-0.68).

7.2 Reward versus Episode

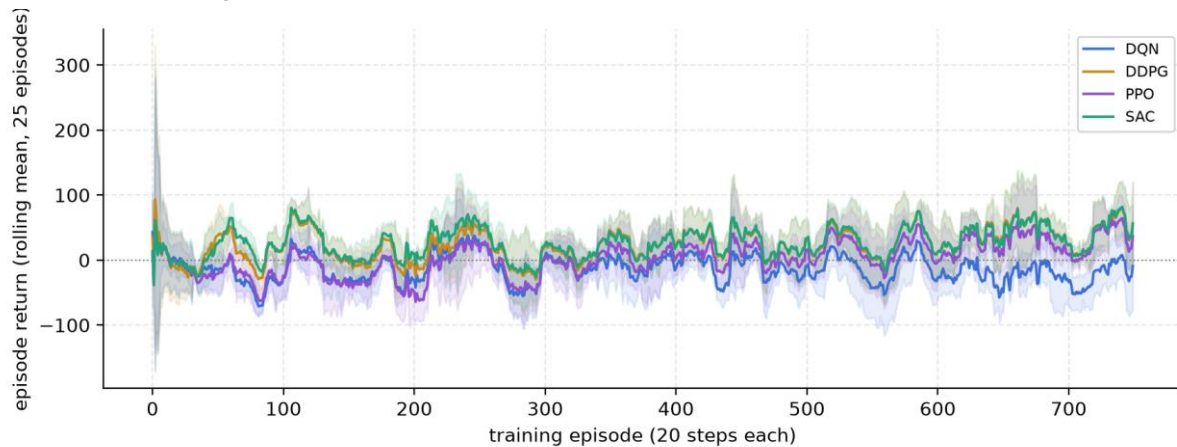


Figure 4. Episode return (sum of the 20 step rewards) against training episode, rolling mean over 25 episodes, mean $\pm$ std across seeds.

Per-episode returns make the ordering visible at the episode scale: the three continuous agents' returns move from slightly negative in the first episodes to clearly positive, SAC and DDPG earliest, PPO later; DQN's returns hover around zero with the widest band, consistent with a policy that was still 43 % random at the end of the budget (Section 9.5).

7.3 Final Policy / Behaviour

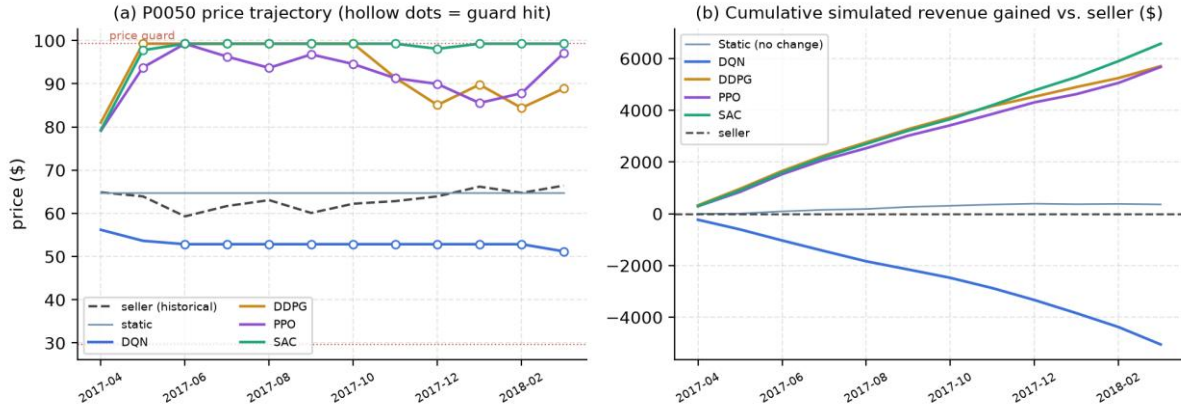


Figure 5. Final policies on held-out product P0050 (furniture), mean over seeds: (a) monthly price chosen by each agent against the seller's actual price, hollow markers where the price guard was hit; (b) cumulative simulated revenue relative to the seller's own prices.

The learned behaviours differ qualitatively. SAC and DDPG raise the price from the entering \$77 to the guard within one or two months and hold it; PPO also moves up but oscillates between \$85 and \$99, reflecting its still-improving policy; DQN moves the opposite way, cutting to the lower guard. Under the fitted demand model this product is inelastic, so the upward policies gain revenue every month while DQN loses it: over twelve months SAC finishes about \$6,900 ahead of the seller and DQN about \$5,000 behind. The hollow markers show that the continuous agents are constrained by the price guard, not by their own preference — a point taken up in Section 9.

7.4 Performance Metrics

Six metrics are reported (definitions in Table 10). Table 9 gives their values for the four approaches and the best on each.

Table 9. Performance metrics for the four approaches (mean over seeds unless stated).

Metric	DQN	DDPG	PPO	SAC	Best
M1 Mean held-out DRCR	0.0313	0.0748	0.0778	0.0822	SAC
M2 Std of held-out DRCR across seeds (lower = more stable)	0.0236	0.0044	0.0046	0.0009	SAC
M3 Simulated revenue uplift vs. seller (%)	-2.97	+3.00	+2.97	+3.92	SAC
M4 Training-tail mean reward (last 20 % of steps)	-0.95 ± 0.80	2.30 ± 0.67	1.61 ± 0.84	2.32 ± 0.67	SAC
M5 Convergence step (running mean within ± 0.25 of final)	4,040	12,355	13,020	11,319	DQN
M6 Guard-hit rate on held-out decisions (%)	71.2	63.3	45.0	62.8	— (behavioural)
M7 Training throughput (env steps / s)	109	59	90	44	DQN

7.5 Comparison Charts

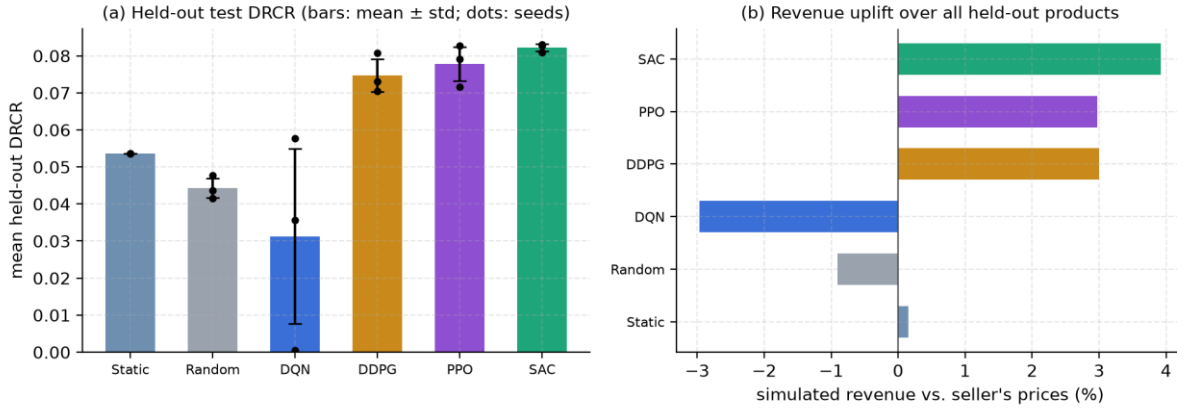


Figure 6. Comparative performance including baselines: (a) held-out DRCR, bars = mean $\pm$ std across seeds, dots = individual seeds; (b) simulated revenue uplift over the seller's prices summed over all held-out products.

7.6 Key Observations

- SAC achieves the highest held-out DRCR (0.0822) and the tightest cross-seed spread (0.0009); its three seeds land within 0.002 of each other.
- All three continuous-action agents beat both the no-change (0.0537) and random (0.0443) baselines and the seller's own prices (uplift +3.0 % to +3.9 %).
- DQN (0.0313) is below the no-change baseline and has the largest spread (0.0236); one seed scored 0.0005.
- DDPG and SAC converge fastest (running mean settled by steps 12,355 and 11,319); PPO's running mean was still moving at the end of the budget.
- Guard-hit rates are high for the continuous agents (DDPG 63 %, SAC 63 %, PPO 45 %) and for DQN (71 %, at the lower bound): the policies want to move further than the guard allows.
- Throughput ranks inversely with model size: DQN 109 steps/s (22,539 parameters) versus SAC 44 steps/s (64,132 parameters); a 15,000-step SAC run takes about 6 minutes on the laptop CPU.

8. Comparative Analysis

Table 10 defines the metrics and repeats their values; the subsections interpret them.

Table 10. Metric definitions and values.

Metric	Definition	DQN	DDPG	PPO	SAC
M1 Held-out DRCR	Mean per-month DRCR over the held-out products and seeds; the trained objective	0.0313	0.0748	0.0778	0.0822
M2 Stability	Standard deviation of M1 across the three seeds	0.0236	0.0044	0.0046	0.0009
M3 Revenue uplift	(policy revenue – seller revenue) / seller revenue over all held-out product-months, both through the demand model	-2.97 %	+3.00 %	+2.97 %	+3.92 %
M4 Tail reward	Mean training reward over the last 20 % of steps ($100 \times$ DRCR scale)	-0.95	2.30	1.61	2.32
M5 Convergence	First step after which the cross-seed running-mean reward stays within ± 0.25 of its final value	4,040	12,355	13,020	11,319
M7 Throughput	Environment steps per second including learning updates (CPU, single thread)	109	59	90	44

8.1 Convergence

Convergence was measured on the running mean of the training reward averaged over seeds: the first step after which it never again leaves a ± 0.25 band around its final value. By this measure DDPG (step 12,355) and SAC (step 11,319) converged fastest, DQN (step 4,040) settled but at a negative level, and PPO (step 13,020) settled last because its running mean was still rising. Off-policy replay lets DDPG and SAC reuse every transition many times; PPO sees each transition once and updated only 14 times in the whole budget.

8.2 Reward and Policy Quality

On the held-out products SAC's policy is the best by every quality metric: highest DRCR, highest uplift, tightest spread. DDPG reaches the same training level (M4 2.30 vs 2.32) but generalises with roughly five times the seed-to-seed variance, which is the practical cost of its externally scheduled exploration noise. PPO's policy is competitive (M1 0.0778) and hits the guard least often (45 %), i.e. it moves prices more cautiously, consistent with its clipped updates. DQN's policy is the only one that lowered prices, and it lost revenue on every aggregate measure.

8.3 Computational Performance

Table 11. Computational cost measured over 1,500 training steps on the laptop CPU (environment step alone: 3.1 ms).

Approach	Parameters	Steps / s	Seconds per 1,000 steps	Inference (μ s / action)	Networks updated per step
DQN	22,539	109	9.2	198	1 (Q)
DDPG	42,626	59	17.0	461	2 (actor, critic)
PPO	42,499	90	11.2	1662	2, but only every 1,024 steps
SAC	64,132	44	22.9	307	4 (actor, 2 critics, α)

Cost scales with the number of networks updated per step. DQN is cheapest; SAC is the most expensive per step (four networks) yet still trains a full run in well under an hour. PPO's per-step cost is low because it batches its updates, but it needs more steps to reach the same policy quality, so its wall-clock to a given quality is not lower.

8.4 Overall Best Approach

SAC is the best approach on the evidence: first on M1, M2 and M3, joint first on M4, second on M5, and its per-seed DRCR values (0.0828, 0.0830, 0.0809) do not overlap with any other algorithm's range. The margin over DDPG on the mean is modest (+0.0075); the decisive difference is reliability. Its higher compute cost is immaterial at this scale.

8.5 Strengths and Weaknesses

Table 12. Strengths and weaknesses of the four approaches.

Approach	Strengths	Weaknesses	Best Use	Observed Performance
DQN	simple, cheapest, stable value learning	5 % price resolution; exploration schedule must be tuned to the budget	few, coarse price tiers; very large SKU counts	M1 0.0313; below baselines in this run
DDPG	exact price; fast convergence; matches SAC's training level	noise schedule hand-tuned; higher variance across seeds	continuous pricing with careful tuning	M1 0.0748; std 0.0044
PPO	robust bounded updates; cautious price moves; no replay memory	sample-inefficient; slowest to converge	noisy rewards with a large step budget	M1 0.0778; still improving at 15,000 steps
SAC	best DRCR and uplift; most stable; exploration self-tuned	four networks; slowest per step	continuous pricing, especially data-poor SKUs	M1 0.0822; std 0.0009

9. Discussion

9.1 Performance Differences

Three design choices explain the ranking. Continuous actions beat discrete ones because the revenue-maximising move is often a fine adjustment that an 11-bucket grid cannot express; this reproduces Liu et al.'s DDPG-over-DQN result [1]. Entropy-regularised exploration beats a hand-scheduled noise process on reliability: SAC and DDPG learn the same thing on average, but SAC's exploration adapts per state through α while DDPG's decays on a clock regardless of what the policy still needs to learn. Off-policy replay beats on-policy updates on sample efficiency within a fixed budget: PPO's clipped objective did what it promises — its policy moved smoothly and conservatively — but with one update per 1,024 steps it had not finished learning.

9.2 Effect of Environment

Stochasticity in the market context makes the per-step reward noisy and forces every learner to average over many products; this is why replay-based methods, which average over the whole buffer, are at an advantage. Partial observability (the agent never sees the demand function) makes the rolling-history features important; ablating them was outside scope but they carry the product-identity information one shared network needs. The continuous, bounded action space with a guard shaped behaviour strongly: because the fitted demand is inelastic ($\beta = -0.89$), revenue rises with price up to the guard for most products, so the continuous agents learned “move up to the boundary” and were clipped on 45–63 % of decisions. A more elastic market would produce interior optima and different policies.

9.3 Exploration versus Exploitation

Each algorithm explores differently and the differences are visible in the results. DQN's ϵ -greedy schedule was designed for a 25,000-step run and the budget was 15,000, so its checkpoints were still exploring 43 % of the time when saved — its evaluation policy ($\epsilon = 0$) is therefore the greedy policy of an under-trained Q-function, which explains both its low mean and its high variance. DDPG's OU noise anneals from 0.20 to 0.05 on the same 25,000-step clock, so at 15,000 steps it was still moderately noisy, yet its deterministic evaluation policy was already good; DDPG is less hurt because its actor, not its exploration, defines the evaluated behaviour. PPO and SAC explore through their policies' own randomness, which shrinks as the policy becomes confident; SAC additionally raises α whenever entropy falls below the target, which kept it exploring in states it had seen rarely. That is the mechanism behind its tight cross-seed spread.

9.4 Practical Interpretation

For a marketplace, the result says: prefer a continuous-action, entropy-regularised agent, evaluate it on products it never trained on, and report seed-to-seed variance, because a policy that is right on average but unreliable across retrainings is not deployable. It also says that the demand model is the product: the agents are only as good as the counterfactual they are trained against, so its validation ($R^2 = 0.94$ on a time-based holdout), its monotonicity guarantee and its guard rails deserve as much engineering as the learners. In this simulator the winning policy is “raise the price to the allowed limit”; on a real market that recommendation stands only if the true elasticity is as low as the fitted one, which is a question for a field test such as [1], not for simulation.

9.5 Limitations

- DQN is under-trained, not only bucket-limited: its ϵ schedule (25,000 steps) exceeds the 15,000-step budget. The bucket penalty is real but its size is overstated here; a 50,000-step run (one command in the repository) removes the confound.
- The demand model under-estimates the elasticity that generated the synthetic data (fitted $\beta = -0.89$ against 1.0–2.5); monthly price moves of about 3 % give little within-product variation and the ridge penalty

shrinks the slope. The simulator is therefore less price-sensitive than the data-generating process, which favours price increases.

- The data are synthetic (schema-identical to the Kaggle dataset, with a known demand law). This is what allows the simulator to be validated, but every number would change on real data; the method and the code would not.
- 20 held-out products and 3 seeds are enough to rank the algorithms, not to give tight confidence intervals; the per-seed points in Figure 6 show the honest spread.
- Competitors are exogenous: they do not react to the agent's price. A reacting competitor (as in [2]) would change the optimal policy and could create collusive or price-war dynamics.
- The traffic proxy (category units sold) is coarser than the page-visit counts used in [1].

10. Innovation Component and Future Extension

Selected innovation: Digital Twin + RL, extended with Offline RL for cold-start products.

The environment built here is already a data-driven digital twin of the market: the real context is replayed and the demand response is a validated model with guard rails, so policies can be trained and compared safely before any live exposure. The proposed extension addresses the problem Liu et al. solve by pre-training on human prices [1] — products with too little history to learn from scratch — with the modern offline-RL toolkit (Figure 7). Offline algorithms such as Conservative Q-Learning (CQL) or Implicit Q-Learning (IQL) learn directly from the logged (state, human price, outcome) tuples of all products, penalising actions far from what was ever tried; the resulting policy initialises SAC, which then fine-tunes inside the twin under the same guard rails. This gives every new SKU a sensible starting policy, keeps the exploration of live customers to a minimum, and turns the guard rails from a training safeguard into a safe-RL constraint that the fine-tuning stage must respect.

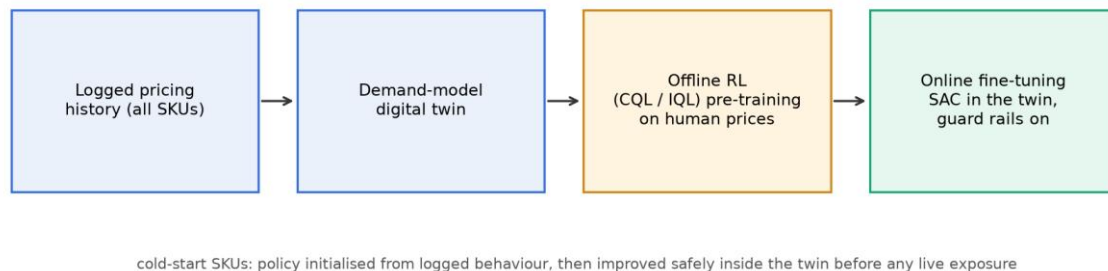


Figure 7. Proposed extension: offline RL pre-training from logged human prices, followed by safe online fine-tuning of SAC inside the demand-model digital twin.

Expected improvements: faster convergence for data-poor SKUs (the policy starts near human behaviour rather than at random), lower variance across retrainings (the same initialisation for every seed), and an explicit safety boundary. Evaluation would reuse this project's protocol: held-out products, several seeds, DRCR and revenue uplift against the seller's own prices.

11. Conclusion

This project turned the team's conceptual comparison of four reinforcement-learning algorithms for dynamic pricing into a measured one. A pricing MDP faithful to Liu et al.'s Tmall system [1] — a 35-feature product-

month state, a bounded price move, and DRCR as reward — was implemented as a Gymnasium environment whose transition and reward come from a validated, guarded counterfactual demand model. DQN, DDPG, PPO and SAC were implemented from first principles, verified on standard benchmarks, trained under an identical budget of 15,000 steps for 3 seeds, and evaluated on 20 held-out products. SAC was the best approach: highest held-out DRCR (0.0822), highest simulated revenue uplift (+3.9 %) and the most reliable across seeds. The practical significance is twofold: continuous, entropy-regularised control is the right tool for this problem, and a validated demand simulator is the precondition for any such comparison on logged data.

11.1 Key Findings

- Continuous-action agents (DDPG 0.0748, PPO 0.0778, SAC 0.0822) outperform the discrete DQN (0.0313) and both baselines, reproducing the paper’s DDPG-over-DQN result.
- SAC’s self-tuned entropy exploration gives the same training level as DDPG with a five-fold tighter held-out spread (0.0009 vs 0.0044).
- PPO’s clipped updates produce the most cautious pricing (lowest guard-hit rate, 45 %) but need a larger budget: its running mean was still rising at 15,000 steps.
- The counterfactual demand model must be identified within product and guarded; a pooled fit produced a wrong-sign elasticity, and an unguarded model invites reward hacking.

11.2 Future Scope

Re-run at the full 50,000-step budget with five seeds to remove DQN’s schedule confound and tighten intervals; replace the synthetic panel with the real Kaggle data (no code changes required); allow per-category elasticity in the demand baseline and a richer traffic proxy; log training losses (now supported) to compare critic stability directly; add reacting competitors; and implement the offline-RL pre-training of Section 10.

12. References

- [1] J. Liu, Y. Zhang, X. Wang, Y. Deng, and X. Wu, “Dynamic pricing on e-commerce platform with deep reinforcement learning: A field experiment,” arXiv preprint arXiv:1912.02572, 2019.
- [2] A. Kastius and R. Schlosser, “Dynamic pricing under competition using reinforcement learning,” *Journal of Revenue and Pricing Management*, vol. 21, pp. 50–63, 2022, doi: 10.1057/s41272-021-00285-3.
- [3] R. Rana and F. S. Oliveira, “Real-time dynamic pricing in a non-stationary environment using model-free reinforcement learning,” *Omega*, vol. 47, pp. 116–126, 2014, doi: 10.1016/j.omega.2013.10.004.
- [4] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [5] T. P. Lillicrap et al., “Continuous control with deep reinforcement learning,” arXiv preprint arXiv:1509.02971, 2015.
- [6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” arXiv preprint arXiv:1707.06347, 2017.
- [7] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *Proc. 35th Int. Conf. Machine Learning (ICML)*, Stockholm, Sweden, 2018, pp. 1861–1870.
- [8] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [9] C. J. C. H. Watkins, “Learning from delayed rewards,” Ph.D. dissertation, King’s College, Cambridge, U.K., 1989.
- [10] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” arXiv preprint arXiv:1506.02438, 2015.
- [11] M. Towers et al., “Gymnasium: A standard interface for reinforcement learning environments,” arXiv preprint arXiv:2407.17032, 2024.

- [12] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in Advances in Neural Information Processing Systems 32, 2019, pp. 8024–8035.
- [13] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [14] Siddharshan, “Retail Price Optimization,” Kaggle dataset (derived from the Olist Brazilian e-commerce dataset). [Online]. Available: <https://www.kaggle.com/datasets/siddharshan/retail-price-optimization> (accessed Sep. 2026).

Appendix A: Project Resources

- Repository (code, configs, trained checkpoints' logs, dashboard, this report's generator): <https://github.com/Feliz7000/rl-dynamic-pricing-comparison>
- Interactive dashboard: docs/index.html in the repository (single file; opens in any browser) — tabs Simulate, What-if, Training, Results.
- Live application: streamlit run app/streamlit_app.py — evaluates the demand model and the four checkpoints on demand.
- Data: synthetic panel from scripts/generate_synthetic_data.py (schema of [14]); scripts/download_data.py fetches the real dataset into data/raw/.
- Reproduction: python scripts/fit_demand_model.py; TOTAL_STEPS=15000 SEEDS="0 1 2" bash scripts/run_full_comparison.sh; python scripts/build_dashboard.py; python scripts/build_report.py.
- Tests: pytest tests/ (data loader, demand model, environment) and tests/test_agents_smoke.py (each algorithm on CartPole-v1 / Pendulum-v1).

Appendix B: Additional Results

Table 13 gives the simulated revenue uplift over the seller's prices for every held-out product (mean over seeds), the per-product view behind Figure 6(b).

Table 13. Per-product simulated revenue uplift vs. the seller's own prices on the held-out set (sorted by SAC).

Product	Category	Seller revenue (model)	DQN	DDPG	PPO	SAC	Best
P0050	furniture	\$131,374	-3.8 %	+4.3 %	+4.3 %	+5.0 %	SAC
P0048	sports	\$101,846	-3.5 %	+4.1 %	+4.6 %	+4.9 %	SAC
P0047	sports	\$40,710	+0.2 %	+1.0 %	+3.9 %	+4.5 %	SAC
P0004	electronics	\$28,627	-6.9 %	+4.2 %	+3.0 %	+4.3 %	SAC
P0022	fashion	\$37,049	-1.6 %	+1.8 %	+4.5 %	+4.2 %	PPO
P0016	home appliances	\$81,099	+0.3 %	+4.4 %	+1.7 %	+4.2 %	DDPG
P0014	home appliances	\$20,634	-3.6 %	+4.1 %	+3.0 %	+4.0 %	DDPG
P0058	groceries	\$31,104	-3.1 %	+4.0 %	+3.1 %	+3.8 %	DDPG
P0051	furniture	\$34,617	-0.5 %	+1.0 %	+3.8 %	+3.7 %	PPO
P0077	garden	\$32,536	-5.5 %	+1.5 %	+2.7 %	+3.7 %	SAC
P0035	toys	\$44,535	-0.4 %	+3.5 %	+2.1 %	+3.5 %	SAC
P0039	toys	\$46,515	-1.1 %	+3.0 %	+0.7 %	+3.3 %	SAC
P0021	fashion	\$39,064	-6.8 %	+1.7 %	+2.3 %	+3.2 %	SAC
P0003	electronics	\$9,738	-6.0 %	-0.7 %	+2.6 %	+3.0 %	SAC
P0057	groceries	\$16,377	-6.3 %	+4.1 %	+2.9 %	+2.8 %	DDPG
P0029	beauty	\$15,385	-3.1 %	+1.1 %	+2.3 %	+2.6 %	SAC
P0071	books	\$24,421	-6.6 %	+0.5 %	-0.1 %	+1.7 %	SAC
P0067	books	\$16,980	-6.2 %	-0.0 %	+0.6 %	+1.0 %	SAC
P0027	beauty	\$8,915	-1.3 %	-0.8 %	-0.5 %	-0.5 %	PPO
P0080	garden	\$5,933	-1.2 %	-1.2 %	-2.2 %	-0.9 %	SAC

Per-seed held-out DRCR: Static (no change): 0.0537, 0.0537, 0.0537; Random: 0.0437, 0.0415, 0.0478; DQN: 0.0005, 0.0577, 0.0357; DDPG: 0.0808, 0.0731, 0.0704; PPO: 0.0791, 0.0716, 0.0827; SAC: 0.0828, 0.0830, 0.0809.